

Guía completa: Formulario de mantenimiento para configuración de mora por punto de venta

Vamos a crear una tabla sencilla, poblarla con datos de ejemplo y construir un formulario completo en Oracle Forms que permita darle mantenimiento (crear, consultar, modificar y eliminar).



Paso 1: Crear la tabla en Oracle

sql

```
CREATE TABLE cfg_mora_punto_venta (  
    id_config          NUMBER PRIMARY KEY,  
    id_punto_venta     NUMBER NOT NULL,  
    cantidad           NUMBER(5) NOT NULL,  
    unidad             VARCHAR2(10) NOT NULL,  
    estado             VARCHAR2(10) DEFAULT 'ACTIVO' NOT NULL,  
    fecha_creacion     DATE DEFAULT SYSDATE,  
    usuario_creacion   VARCHAR2(50),  
    CONSTRAINT fk_cfg_pv FOREIGN KEY (id_punto_venta)  
        REFERENCES punto_venta(id_punto_venta),  
    CONSTRAINT ck_cfg_unidad CHECK (unidad IN ('HORAS', 'DIAS')),  
    CONSTRAINT ck_cfg_estado CHECK (estado IN ('ACTIVO', 'INACTIVO')),  
    CONSTRAINT ck_cfg_cantidad CHECK (cantidad > 0)  
);
```

```
CREATE SEQUENCE seq_cfg_mora_pv START WITH 1 INCREMENT BY 1;
```

Datos de ejemplo

sql

```
INSERT INTO cfg_mora_punto_venta  
    (id_config, id_punto_venta, cantidad, unidad, usuario_creacion)  
VALUES (seq_cfg_mora_pv.NEXTVAL, 10000, 24, 'HORAS', 'ADMIN');  
INSERT INTO cfg_mora_punto_venta  
    (id_config, id_punto_venta, cantidad, unidad, usuario_creacion)
```

```
VALUES (seq_cfg_mora_pv.NEXTVAL, 10001, 30, 'DIAS', 'ADMIN');
INSERT INTO cfg_mora_punto_venta
(id_config, id_punto_venta, cantidad, unidad, usuario_creacion)
VALUES (seq_cfg_mora_pv.NEXTVAL, 10002, 48, 'HORAS', 'ADMIN');
INSERT INTO cfg_mora_punto_venta
(id_config, id_punto_venta, cantidad, unidad, usuario_creacion)
VALUES (seq_cfg_mora_pv.NEXTVAL, 10003, 15, 'DIAS', 'ADMIN');
COMMIT;
```

Vista de apoyo (para mostrar el nombre del PV en el formulario)

sql

```
CREATE OR REPLACE VIEW v_cfg_mora_pv AS
SELECT c.id_config,
       c.id_punto_venta,
       pv.nombre_pv,
       c.cantidad,
       c.unidad,
       c.estado,
       c.fecha_creacion,
       c.usuario_creacion
FROM   cfg_mora_punto_venta c
JOIN   punto_venta pv ON pv.id_punto_venta = c.id_punto_venta;
```



Paso 2: Crear el módulo en Forms Builder

1. Abre Forms Builder.
 2. **File** → **New** → **Form**.
 3. Guarda como `FOR_CFG_MORA_PV.fmb`.
-



Paso 3: Crear la ventana y el canvas

3.1 Ventana

1.Clic derecho sobre **Windows** → **Create**.

2.Property Palette:

- Name:** WIN_CFG
- Title:** Configuración de Mora por Punto de Venta
- Width:** 900
- Height:** 600
- Visible:** Yes

3.2 Canvas

1.Clic derecho sobre **Canvases** → **Create** → tipo **Content**.

2.Property Palette:

- Name:** CVS_CFG
- Window:** WIN_CFG
- Visible:** Yes
- Width:** 900
- Height:** 600

3.En la ventana WIN_CFG, pon **Primary Canvas** = CVS_CFG.



Paso 4: Crear el bloque de datos

4.1 Crear el bloque manualmente

1.Clic derecho sobre **Data Blocks** → **Create** → "**Build a new data block manually**" → OK.

2.Property Palette:

- Name:** BLK_CFG
- Database Data Block:** Yes
- Query Data Source Type:** Table
- Query Data Source Name:** CFG_MORA_PUNTO_VENTA

- Query All Records: No
- Query Array Size: 10
- Number of Records Displayed: 10
- Enforce Primary Key: Yes
- Key Mode: Automatic
- Update Allowed: Yes
- Insert Allowed: Yes
- Delete Allowed: Yes

4.2 Crear los ítems

Crea cada ítem con **clic derecho sobre BLK_CFG → Create Item**. Todos con **Canvas = CVS_CFG**.

Item 1: ID_CONFIG

Propiedad	Valor
Name	ID_CONFIG
Item Type	Text Item
Data Type	Number
Database Item	Yes
Prompt	ID
Width	60
Height	20
Update Allowed	No
Insert Allowed	No
Required	No
Navigable	No
Enabled	No
Background Color	Gris (para que parezca no editable)

Item 2: ID_PUNTO_VENTA

Propiedad	Valor
Name	ID_PUNTO_VENTA
Item Type	Text Item
Data Type	Number
Database Item	Yes
Prompt	ID Punto de Venta
Width	80
Height	20
Required	Yes
LOV	LOV_PUNTO_VENTA (lo creamos después)
Validate from List	Yes

Item 3: NOMBRE_PV (solo visualización)

Propiedad	Valor
Name	NOMBRE_PV
Item Type	Text Item
Data Type	Char
Maximum Length	150
Database Item	No
Prompt	Nombre PV
Width	200
Height	20
Query Only	Yes
Navigable	No

Item 4: CANTIDAD

Propiedad	Valor
Name	CANTIDAD
Item Type	Text Item
Data Type	Number
Database Item	Yes
Prompt	Cantidad
Width	60
Height	20
Required	Yes
Format Mask	99999

Item 5: UNIDAD

Propiedad	Valor
Name	UNIDAD
Item Type	List Item
Data Type	Char
Database Item	Yes
Prompt	Unidad
Width	80
Height	20
Required	Yes
List Style	Poplist
Elements in List:	HORAS → HORAS, DIAS → DIAS
Initial Value	HORAS

Item 6: ESTADO

Propiedad	Valor
Name	ESTADO
Item Type	List Item
Data Type	Char
Database Item	Yes
Prompt	Estado
Width	80
Height	20
Required	Yes
List Style	Poplist
Elements in List:	ACTIVO → ACTIVO, INACTIVO → INACTIVO
Initial Value	ACTIVO

Item 7: FECHA_CREACION

Propiedad	Valor
Name	FECHA_CREACION
Item Type	Text Item
Data Type	Date
Database Item	Yes
Prompt	Fecha Creación
Width	100
Height	20
Format Mask	DD/MM/YYYY HH24:MI
Update Allowed	No

Estado: [▼]	Fecha: []
Usuario: []	

4. Agrega un **prompt visual** con el texto "**Configuración de Tiempo de Mora por Punto de Venta**" como un ítem Display o directamente en el canvas.

5. Ajusta posiciones (valores aproximados):

Item	X	Y
ID_CONFIG	30	60
ID_PUNTO_VENTA	300	60
NOMBRE_PV	30	110
CANTIDAD	30	160
UNIDAD	300	160
ESTADO	30	210
FECHA_CREACION	300	210
USUARIO_CREACION	30	260

6. Guarda con **Ctrl + S**.



Paso 6: Crear el LOV de Puntos de Venta

Un **LOV (List of Values)** permite al usuario seleccionar un punto de venta de una lista en lugar de escribirlo a mano.

6.1 Crear el Record Group

1. En el Object Navigator: clic derecho sobre **Record Groups** → **Create**.

2. Property Palette:

• **Name:** RG_PUNTO_VENTA

• **Query:**

sql

```
SELECT id_punto_venta, nombre_pv
FROM punto_venta
WHERE estado = 'ACTIVO'
ORDER BY nombre_pv
```

3. Verifica que el **Data Type** de las columnas sea **Number** (para `id_punto_venta`) y **Char** (para `nombre_pv`).

6.2 Crear el LOV

1. Clic derecho sobre **LOVs** → **Create** → te ofrece usar el LOV Wizard. Elige **No** (lo creamos manual).

2. Property Palette:

- **Name:** `LOV_PUNTO_VENTA`
- **Record Group:** `RG_PUNTO_VENTA`
- **Title:** `Seleccione Punto de Venta`
- **Automatic Display:** Yes
- **Automatic Refresh:** Yes
- **Automatic Select:** No
- **Validate from List:** Yes

3. Dentro del LOV, edita las **Column Mappings**:

- Columna 1 (`id_punto_venta`) → Return Item: `BLK_CFG.ID_PUNTO_VENTA`
- Columna 2 (`nombre_pv`) → Display

6.3 Asignar el LOV al ítem

1. Selecciona el ítem `BLK_CFG.ID_PUNTO_VENTA`.

2. Property Palette:

- **LOV:** `LOV_PUNTO_VENTA`
 - **Validate from List:** Yes
-



Paso 7: Triggers

7.1 WHEN-NEW-FORM-INSTANCE (a nivel de módulo)

Clic derecho sobre el módulo → **Create Trigger** → WHEN-NEW-FORM-INSTANCE:

sql

BEGIN

-- Establecer el usuario de la app

:GLOBAL.usuario_app := NVL(GET_APPLICATION_PROPERTY(USERNAME), 'ADMIN');

-- Configurar propiedades de navegación del bloque

SET_BLOCK_PROPERTY('BLK_CFG', INSERT_ALLOWED, PROPERTY_TRUE);

SET_BLOCK_PROPERTY('BLK_CFG', UPDATE_ALLOWED, PROPERTY_TRUE);

SET_BLOCK_PROPERTY('BLK_CFG', DELETE_ALLOWED, PROPERTY_TRUE);

-- Posicionarse en el primer campo editable

GO_ITEM('BLK_CFG.ID_PUNTO_VENTA');

END;

7.2 WHEN-VALIDATE-ITEM en ID_PUNTO_VENTA

Cuando el usuario selecciona o escribe un ID, este trigger busca el nombre del PV:

sql

DECLARE

v_nombre VARCHAR2(150);

BEGIN

IF :BLK_CFG.ID_PUNTO_VENTA IS NULL THEN

:BLK_CFG.NOMBRE_PV := NULL;

RETURN;

END IF;

BEGIN

SELECT nombre_pv

INTO v_nombre

FROM punto_venta

```

        WHERE id_punto_venta = :BLK_CFG.ID_PUNTO_VENTA
        AND estado = 'ACTIVO';

        :BLK_CFG.NOMBRE_PV := v_nombre;
EXCEPTION
    WHEN NO_DATA_FOUND THEN
        :BLK_CFG.NOMBRE_PV := NULL;
        MESSAGE('El punto de venta no existe o está inactivo.');
```

RAISE FORM_TRIGGER_FAILURE;

END;

END;

7.3 PRE-INSERT en el bloque BLK_CFG

Se ejecuta **antes de insertar**. Genera el ID y asigna fecha/usuario:

```

sql

BEGIN
    :BLK_CFG.ID_CONFIG           := seq_cfg_mora_pv.NEXTVAL;
    :BLK_CFG.FECHA_CREACION     := SYSDATE;
    :BLK_CFG.USUARIO_CREACION   := :GLOBAL.usuario_app;
END;
```

7.4 POST-QUERY en el bloque BLK_CFG

Se ejecuta **una vez por fila** recuperada en una consulta. Llena el nombre del PV:

```

sql

DECLARE
    v_nombre VARCHAR2(150);
BEGIN
    BEGIN
        SELECT nombre_pv
        INTO v_nombre
        FROM punto_venta
        WHERE id_punto_venta = :BLK_CFG.ID_PUNTO_VENTA;

        :BLK_CFG.NOMBRE_PV := v_nombre;
```

```

EXCEPTION
    WHEN NO_DATA_FOUND THEN
        :BLK_CFG.NOMBRE_PV := '(no encontrado)';
END;
END;

```

7.5 ON-ERROR en el bloque BLK_CFG

Captura errores de la base de datos y los muestra amigablemente:

```

sql

BEGIN
    IF ERROR_CODE = 40102 THEN -- ORA-01400: cannot insert NULL
        MESSAGE('Debe completar todos los campos obligatorios.');
```

ELSIF ERROR_CODE = 40106 THEN -- ORA-00001: unique constraint violated

```

        MESSAGE('Ya existe una configuración para este punto de venta.');
```

ELSIF ERROR_CODE = 40506 THEN -- ORA-02291: integrity constraint violated (FK)

```

        MESSAGE('El punto de venta seleccionado no existe.');
```

ELSIF ERROR_CODE = 40509 THEN -- ORA-02292: integrity constraint violated (hijo)

```

        MESSAGE('No se puede eliminar: hay registros dependientes.');
```

ELSE

```

        MESSAGE('Error ' || ERROR_CODE || ': ' || ERROR_TEXT);
END IF;
```

RAISE FORM_TRIGGER_FAILURE;

```

END;

```

7.6 KEY-EXIT (opcional, para el botón "Salir")

Clic derecho sobre el módulo → **Create Trigger** → KEY-EXIT:

```

sql

BEGIN
    EXIT_FORM(NO_VALIDATE);
END;

```



Paso 8: Agregar botones de acción (opcional pero recomendado)

Para que el usuario pueda desplazarse entre los registros, crea un **segundo bloque** o usa los botones estándar de la barra de Forms. Si quieres botones propios, crea un bloque `BLK_ACCION` (Control Block) con estos botones:

1. Clic derecho sobre **Data Blocks** → **Create** → **Build manually**.

2. Property Palette:

• **Name:** `BLK_ACCION`

• **Database Data Block:** No

3. Crea los botones:

Nombre	Label	Trigger WHEN-BUTTON-PRESSED
<code>BT_GUARDAR</code>	Guardar	<code>COMMIT_FORM;</code>
<code>BT_CONSULTAR</code>	Consultar	<code>EXECUTE_QUERY;</code>
<code>BT_NUEVO</code>	Nuevo	<code>CREATE_RECORD;</code>
<code>BT_ELIMINAR</code>	Eliminar	<code>DELETE_RECORD; COMMIT_FORM;</code>
<code>BT_SALIR</code>	Salir	<code>EXIT_FORM(NO_VALIDATE);</code>

Todos con **Canvas** = `CVS_CFG` y asignados al canvas.



Paso 9: Integrar al menú existente

9.1 Agregar la opción al menú (`MENU_PRINCIPAL.mmb`)

1. Abre tu menú en Forms Builder.

2. Encuentra el nodo **Menus**.

3. Clic derecho sobre el menú donde quieras agregar la opción → **Create**.

4. Property Palette del nuevo ítem:

• **Name:** `ITEM_CFG_MORA`

•**Label:** Configuración de Mora por PV

•**Command Type:** PL/SQL

•**Command Text:**

```
sql
```

```
CALL_FORM('FOR_CFG_MORA_PV', NO_HIDE, NO_REPLACE, NO_QUERY_ONLY);
```

5.Guarda y compila el menú (Ctrl + K).

9.2 Copiar el .fmx al servidor

1.Compila tu formulario FOR_CFG_MORA_PV.fmb con **Ctrl + K**. Genera FOR_CFG_MORA_PV.fmx.

2.Sube el .fmx al servidor de Forms (usa WinSCP) en la carpeta FORMS_PATH.

3.Sube también el MENU_PRINCIPAL.mmx compilado.



Paso 10: Probar el formulario

1.Compila el formulario (Ctrl + K).

2.Ejecuta (Ctrl + R).

3.Prueba:

•**Consultar:** Presiona el botón "Consultar" y verás los 4 registros de ejemplo.

•**Nuevo:** Presiona "Nuevo", llena los datos, guarda.

•**Modificar:** Busca un registro, cambia la cantidad o unidad, guarda.

•**Eliminar:** Selecciona un registro, presiona "Eliminar", confirma.

•**LOV:** En ID_PUNTO_VENTA, presiona F9 o el botón de LOV para ver la lista.



Resumen de objetos creados

Objeto	Tipo	Ubicación
cfg_mora_punto_venta	Tabla	Oracle DB
seq_cfg_mora_pv	Secuencia	Oracle DB
v_cfg_mora_pv	Vista	Oracle DB
FOR_CFG_MORA_PV.fmb	Formulario	Forms Builder

Objeto	Tipo	Ubicación
BLK_CFG	Data Block	Formulario
BLK_ACCION	Control Block	Formulario
CVS_CFG	Canvas	Formulario
WIN_CFG	Window	Formulario
RG_PUNTO_VENTA	Record Group	Formulario
LOV_PUNTO_VENTA	LOV	Formulario
WHEN-NEW-FORM-INSTANCE	Trigger	Formulario
WHEN-VALIDATE-ITEM	Trigger	ID_PUNTO_VENTA
PRE-INSERT	Trigger	BLK_CFG
POST-QUERY	Trigger	BLK_CFG
ON-ERROR	Trigger	BLK_CFG
KEY-EXIT	Trigger	Formulario

⚠ Cosas a tener en cuenta

- **La vista** `v_cfg_mora_pv` no se usó directamente en el bloque; en su lugar usamos la tabla `cfg_mora_punto_venta` y llenamos `NOMBRE_PV` en el `POST-QUERY`. Esto es más simple y funciona bien.
- **El ID se genera en** `PRE-INSERT`, no en el formulario. Así evitamos problemas si alguien usa la tabla desde otro proceso.
- **El** `ON-ERROR` es clave para que el usuario no vea mensajes crípticos de Oracle.
- **El** `LOV` debe tener el `Record Group` con el `ORDER BY` correcto para que sea agradable.
- **La validación del** `ID_PUNTO_VENTA` en `WHEN-VALIDATE-ITEM` evita que inserten IDs inexistentes.

Con esto tienes todo lo necesario para crear el formulario completo. Si algún trigger o propiedad te da error, pégame el mensaje y lo resolvemos.